A Full-Stack Web Application designed to help wedding planners manage their clients more efficiently, while providing a single location for couples to view and select vendors, venues, and guest lists

Fall Design Presentation: Tennemore Wedding Planning Application

Names:

Team Members:

Michael Weinstein

Weinstmd@mail.uc.edu

Advisor:

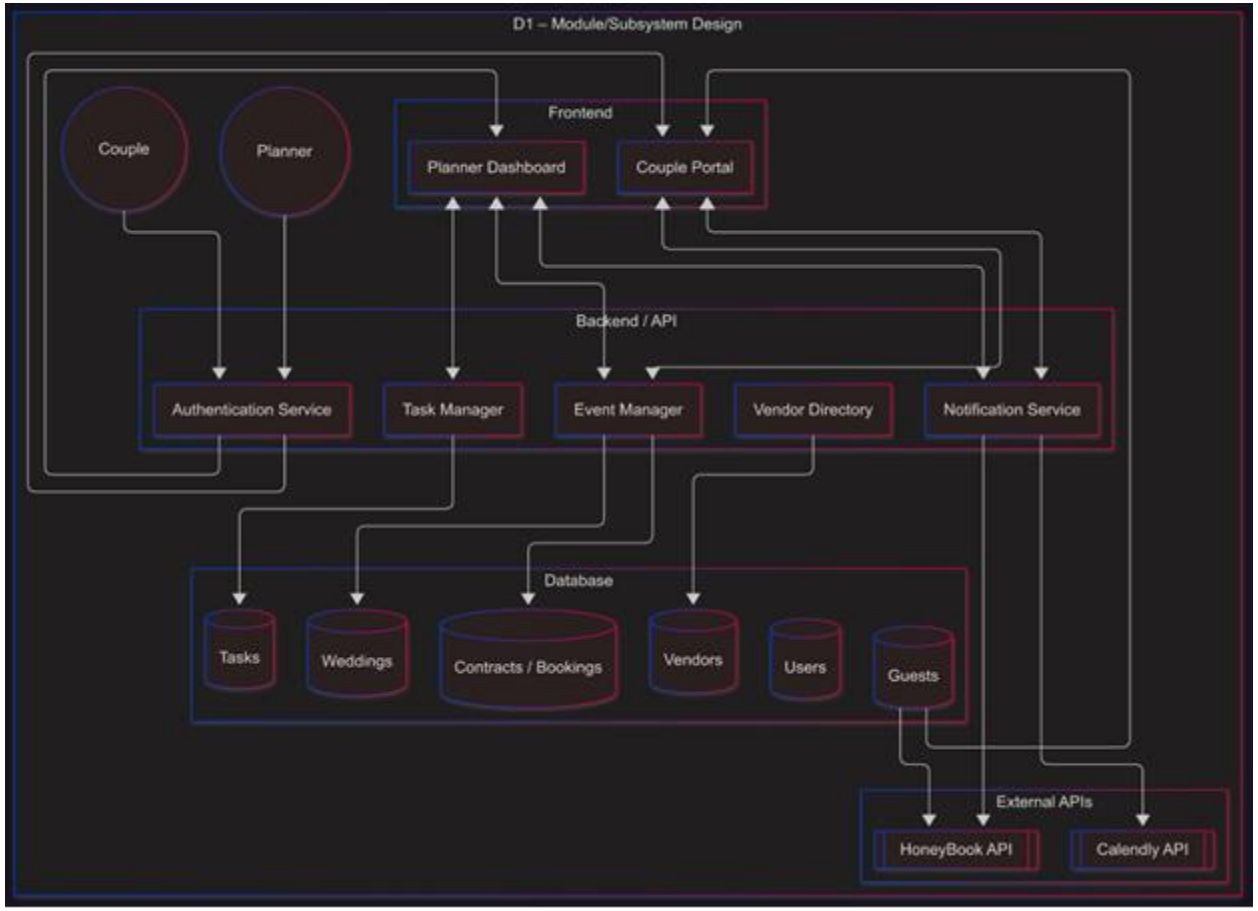
Dr. Will Hawkins

Hawkinwh@ucmail.uc.edu

Project Abstract

Wedding planning often involves fragmented communication and manual scheduling across multiple parties. Tennemore seeks to solve this problem by providing a centralized platform that automates scheduling, streamlines vendor and guest communication, and organizes key planning milestones. Through a responsive React frontend, a serverless Node.js backend, and an integrated database, the system enables real-time updates, task management, and automatic conflict detection.

Design Diagrams

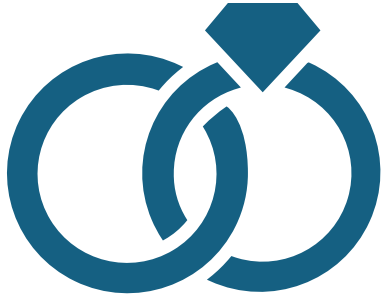


Frontend:
The frontend of our application is the primary method for planners and clients to create, manage, and approve various pre wedding activities, like vendor selection or creating guest lists

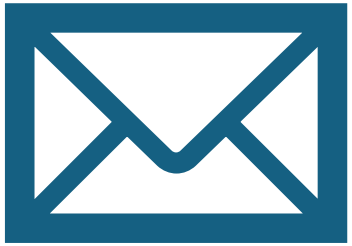
Backend:
The backend is where several third-party APIs will be incorporated into the app to enable scheduling, bookings, and even handle payment processing.

Database:
The database provides persistent storage for various entity types as needed

User Stories



As a couple, I want to receive automated emails with vendor options and booking confirmations so that I can stay informed about the progress of my wedding planning without needing constant check-ins.



As a wedding planner, I want to generate a high-level timeline of upcoming milestones for each wedding so that I can manage multiple clients more efficiently and balance my workload.

Major Project Constraints

Ethical/Security: Need to ensure user's personal data is not exposed while stored in database

Professional: I'm providing a web-hosted application that will require maintenance, which I can't promise I'll be able to deliver. I need to keep very detail documentation to enable future developers to take over with minimal issues.

Personal: I'm building this app for my sister's wedding planning business – if I build the backend API incorrectly, it could dramatically increase the cost of keeping the app up and running.

Review of Project Progress

- The tasks, weddings, and planner portions of the database have been created in a local PostgreSQL server
- The frontend has been created from the vite framework. Further functionality depends on the backend for handling multiple users, password authentication, and task creation.
- The backend so far is only the framework established by the node.js framework, as well as the Express routes that are automatically created during installation. Limited custom routes have been written but need more testing to verify functionality

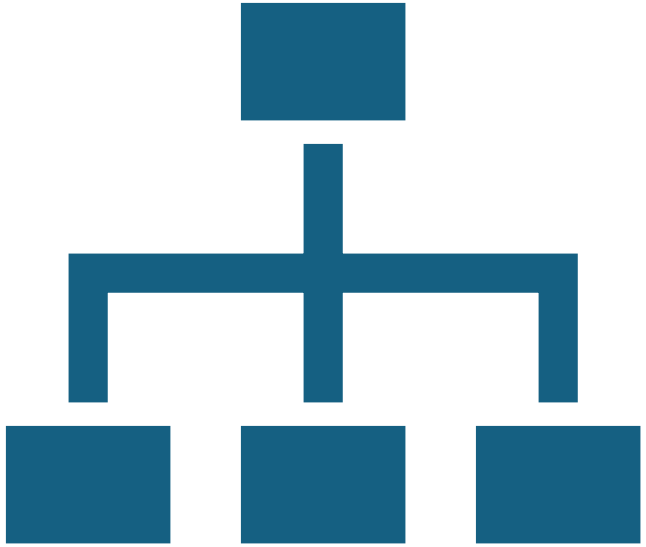
Expected Accomplishments

By the end of the term, I plan to have the following features set up and hosted locally:

- The frontend for the planner dashboard, including the log-in, client management, and vendor screens
- The backend task scheduling and reminders automations, as well as the password management functionality (Ideally with email reset via third-party APIs)
- The vendor database system (client consultation needed)

Division of Work

Since I am the sole member of my project's team, I am responsible for all steps of the project.



Expected Demo at Expo

I look forward to demonstrating a fully functioning application at UC's Design Expo. This will include:

- A modern-looking frontend website for both the couple's portal and planner dashboard
- The backend should be able to automatically backwards plan a wedding given the required details
- The application will be hosted on an online service in accordance with the as-yet-to-be-determined specifications that the Tennemore company will provide.